

Design patterns 2

Soon Phei, Tin

Behavioral Pattern

- Concerned with the assignment of responsibilities between objects, or, encapsulating behavior in an object and delegating requests to it
- Concerned with the way objects are interconnected. The interaction between the objects should be in such a way that they can easily talk to each other and still should be loosely coupled
- The implementation and the client should be loosely coupled in order to avoid hard coding and dependencies

Behavioral Patterns

- **Chain of Responsibilities.** A chain of objects is created to deal with the request so that no request goes back unfulfilled
- **Command.** Command pattern deals with requests by hiding it inside an object as a command and send to the invoker object which then passes it to an appropriate object that can fulfill the request.
- **Interpreter.** Interpreter pattern is used for language or expression evaluation by creating an interface that tells the context for interpretation.
- **Iterator.** Iterator pattern is used to provide sequential access to a number elements present inside a collection object without any relevant information exchange.

Behavioral Patterns

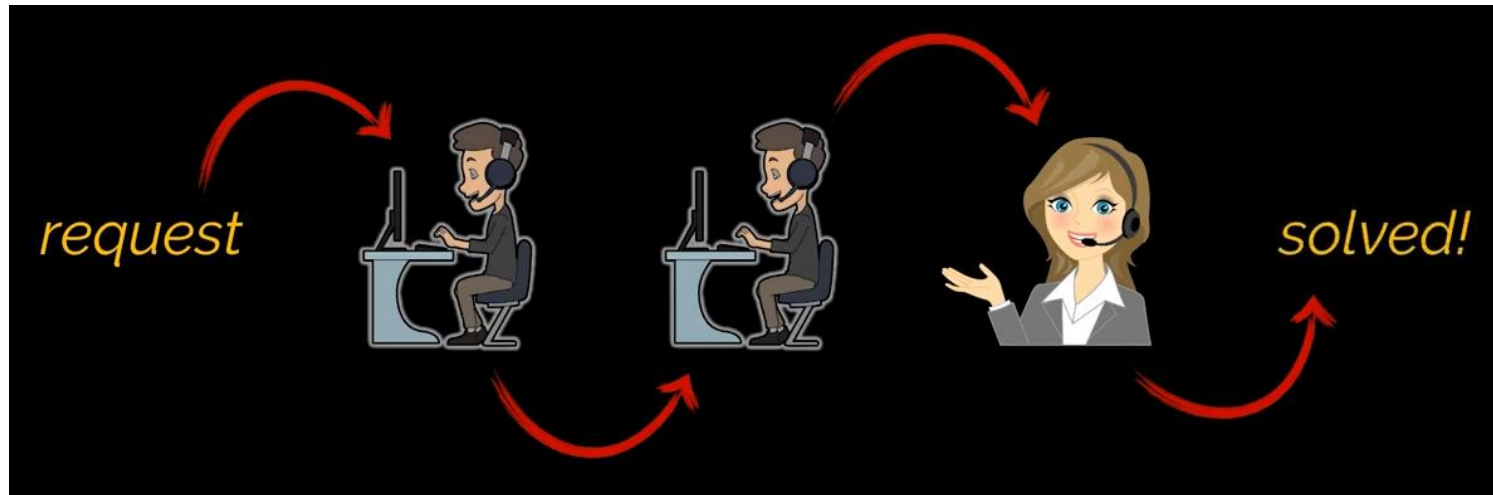
- **State.** In State pattern, the behavior of a class varies with its state and is thus represented by the context object.
- **Strategy.** Strategy pattern deals with the change in class behavior at runtime. The objects consist of strategies and the context object judges the behavior at runtime of each strategy.
- **Visitor.** A Visitor performs a set of operations on an element class and changes its behavior of execution. Thus, the variance in the behavior of element class is dependent on the change in visitor class.

Behavioral Patterns

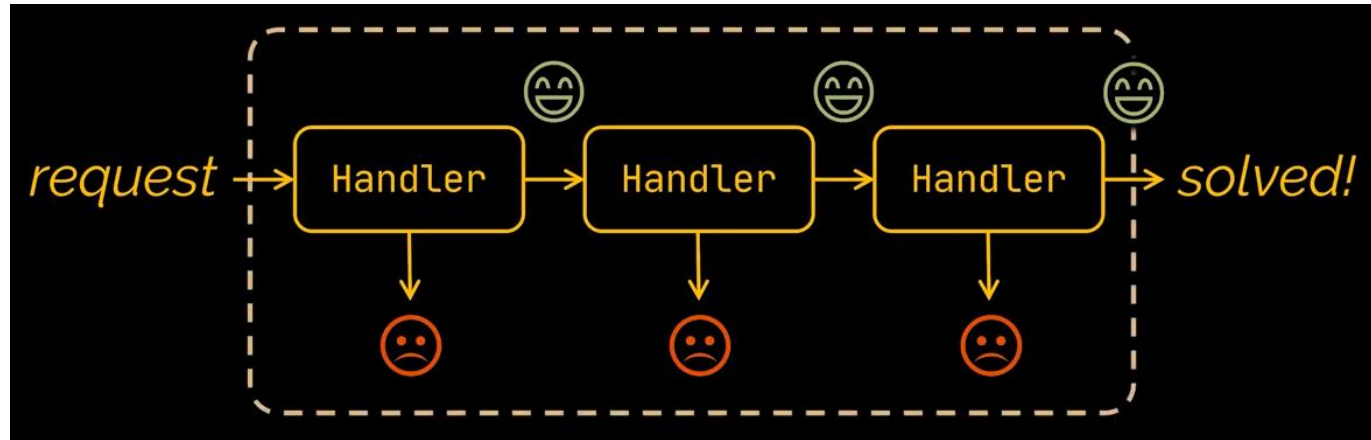
- **Mediator.** Mediator pattern provides easy communication through its mediator class that allows communication for several classes.
- **Observer.** A One-to-Many relationship calls for the need of Observer pattern to check the relative dependencies of objects.

Chain of Responsibilities

- Request route from one service to the other.
- Each service is responsible for one job function.



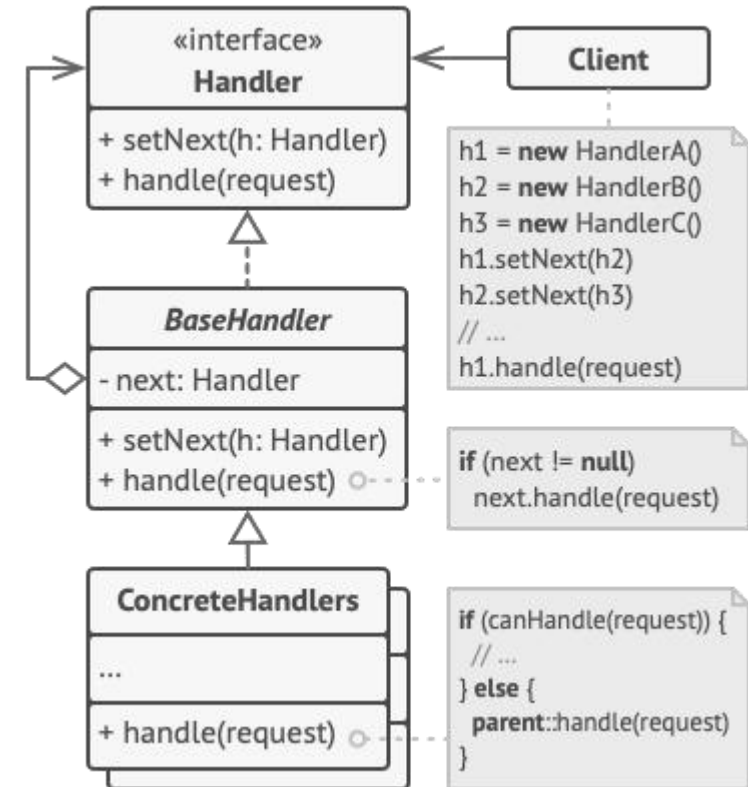
Chain of Responsibilities Pattern



- used to achieve loose coupling in software design where a request from client is passed to a chain of objects to process them.
- Then the object in the chain will decide themselves who will be processing the request and whether the request is required to be sent to the next object in the chain or not.

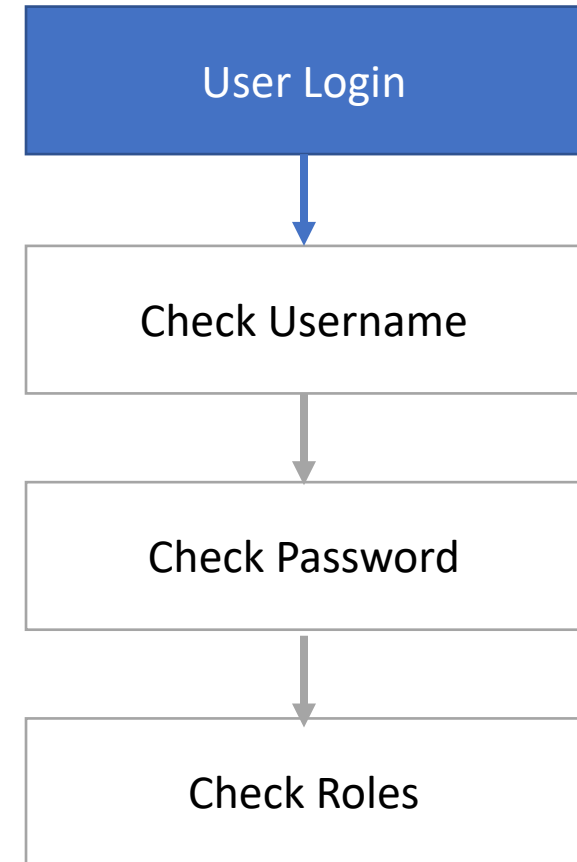
Key Components

- **Handler:** Define the interface for the method for handling request. Sometimes also define the interface for the method to set the next handler.
- **BaseHandler** : This is an optional abstract class. Base class of the concrete handlers. Usually define the boilerplate code common to all concrete handlers.
- **Concrete handlers** : These are actual handlers of the request chained in some sequential order. It contain the actual processing of the requests.
- **Client** : Originator of request and this will access the handler to handle it. It composes the chain of handlers statically or dynamically.

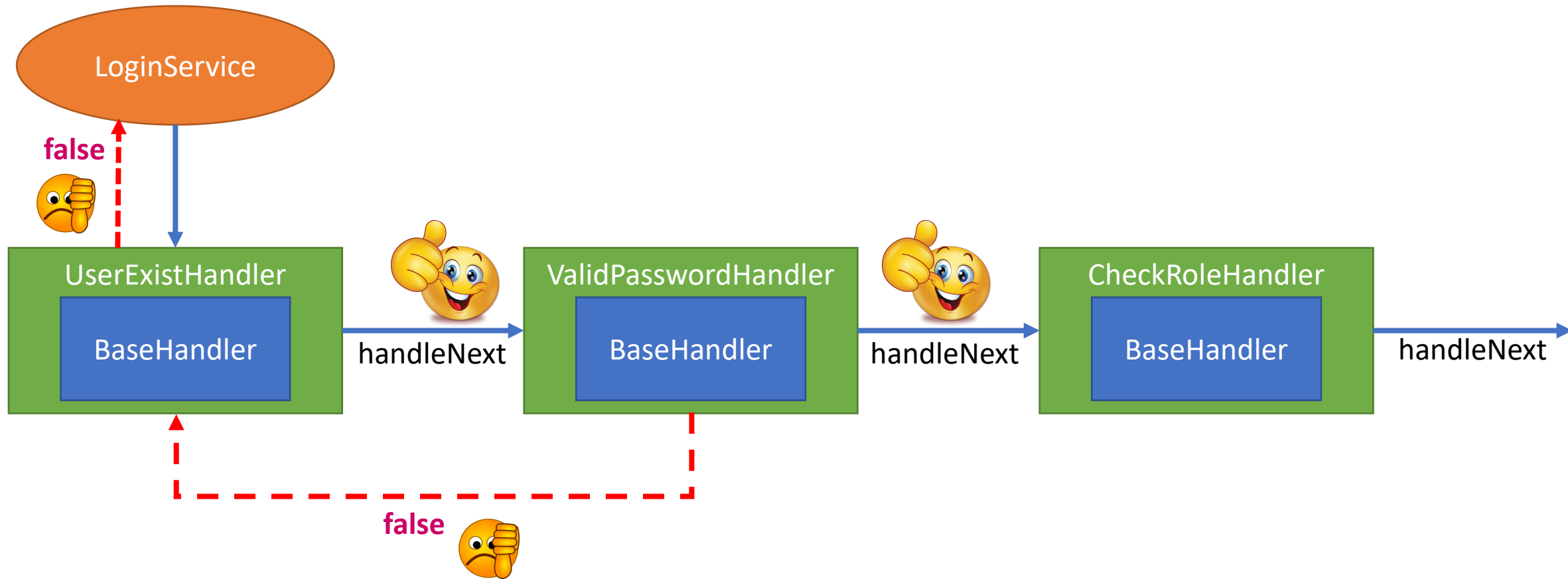


User Authentication and Authorization Example

- The process to authenticate and authorize a user is as follow
 1. Check that the user enter an existing username
 2. Check that the user enter a corresponding password
 3. Check the attached roles of the user
- Use the Chain of Responsibility pattern to design the software.



Implementation



Exercise 1

- Observe the code in the following slides
 - How the implementation in the code different from the design?
 - How do you amend the code to align with the design?

```
public interface Handler {  
    void setNext(Handler handler);  
    void process(User user);  
}
```

```
public class UsernameExistsHandler implements Handler {  
    private Handler next;  
  
    @Override  
    public void setNext(Handler handler) {  
        this.next = handler;  
    }  
  
    @Override  
    public void process(User user) {  
        if (usernameExists(user)) {  
            if (next != null) {  
                next.process(user);  
            }  
        } else {  
            throw new RuntimeException(message: "Username does not exist");  
        }  
    }  
  
    private boolean usernameExists(User user) {  
        // Code to check username exists  
        return true;  
    }  
}
```

```

public class PasswordCorrectHandler implements Handler {
    private Handler next;

    @Override
    public void setNext(Handler handler) {
        this.next = handler;
    }

    @Override
    public void process(User user) {
        if (passwordCorrect(user)) {
            if (next != null) {
                next.process(user);
            }
        } else {
            throw new RuntimeException(message);
        }
    }

    private boolean passwordCorrect(User user) {
        // Code to check password
        return true;
    }
}

```

```

public class RoleExistsHandler implements Handler {
    private Handler next;

    @Override
    public void setNext(Handler handler) {
        this.next = handler;
    }

    @Override
    public void process(User user) {
        if (hasRole(user)) {
            if (next != null) {
                next.process(user);
            }
        } else {
            throw new RuntimeException(message: "User does not have requested roles");
        }
    }

    private boolean hasRole(User user) {
        // Code to check if user has requested roles
        return true;
    }
}

```

```
public class Authentication{
```

Run | Debug

```
public static void main(String[] args) {  
    Handler usernameHandler = new UsernameExistsHandler();  
    Handler passwordHandler = new PasswordCorrectHandler();  
    Handler roleHandler = new RoleExistsHandler();  
  
    usernameHandler.setNext(passwordHandler);  
    passwordHandler.setNext(roleHandler);  
  
    User user = new User("username", "password", "role");  
    usernameHandler.process(user);  
}
```

```
}
```

Important Note

- Client can set the processing order and it will send the request to the first handler in the chain.
- Each handler in the chain will have its own implementation to process the request. It will then send the remaining tasks to the next handler in the chain for further processing.
- Every handlers in the chain should have reference to the next handler in chain to forward the request to.
- Creating the chain carefully is very important otherwise there might be a case that the request will never be forwarded to a particular handler or there are no handler in the chain who are able to handle the request.
- Chain of Responsibility design pattern is good to achieve loose coupling, but it comes with the trade-off of having a lot of implementation classes and maintenance problems if most of the code is common in all the implementations.

Benefits

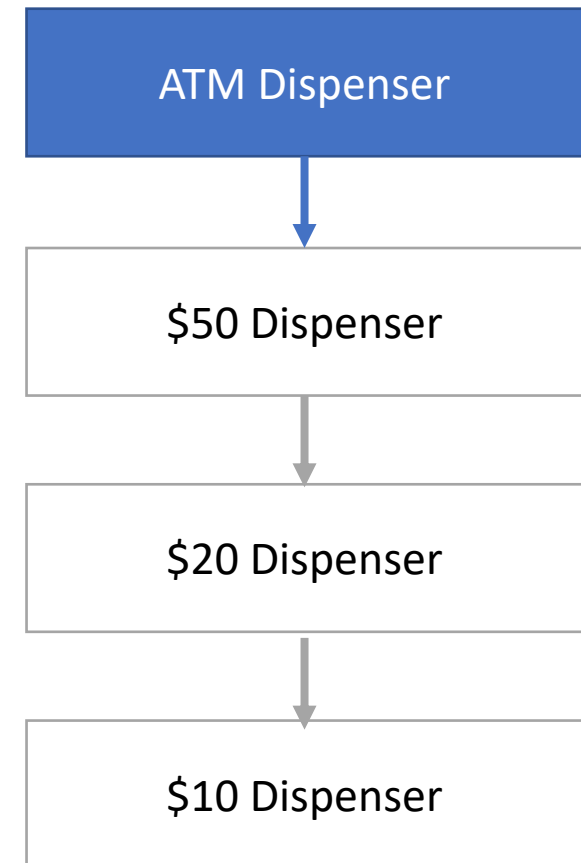
- **Reduced Coupling:** The pattern frees an object from knowing which other object handles a request.
- **Added Flexibility:** You can add, remove, or change the order of responsibility dynamically or at runtime.
- **Clear Sequential Process:** Chain of Responsibility pattern provides a very clear and explicit control over the sequence.
- **Simplicity:** Avoid using complicated code with multiple conditionals to handle different types of requests, the pattern chains the receiving objects and passes the request along the chain until an object handles it.
- **Separation of Concerns:** Each handler is independent of others and doesn't need to know about the inner workings of other handlers.

Application of Chain Of Responsibility

- Use the Chain of Responsibility pattern when your program is expected to process different kinds of requests in various ways, but the exact types of requests and their sequences are unknown beforehand.
- Use the pattern when it's essential to execute several handlers in a particular order.
- Use the CoR pattern when the set of handlers and their order are supposed to change at runtime.

ATM Problem

- In ATM dispense machine, the user enters the amount to be dispensed and the machine dispense amount in terms of defined currency bills such as \$50, \$20, \$10 etc.
- If the user enters an amount that is not multiples of 10, it throws error.
- We will use Chain of Responsibility pattern to implement this solution. The chain will process the request in the same order as below image.
- Note that we can implement this solution easily in a single program itself but then the complexity will increase, and the solution will be tightly coupled.



Implementation

- Given the below interface, implement the ATM program using Chain of Responsibility design pattern

```
1  public interface Handler{  
2      public Handler setNextHandler(Handler handler);  
3  
4      public void dispense(double amount);  
5  }
```

- Implementation tips
 - For each handler, in the *dispense* method, if the amount to dispense is greater than the amount the handler need to dispense, then
 - Calculate the number of notes to dispense
 - Calculate the remaining amount and call the next handler to dispense the remaining amount
 - Remember to check if there is any *next handler*

Exercise 2

- Consider an example of handling HTTP requests on a web server. In a web server, multiple types of operations need to be performed on a HTTP request. These can include authentication checks, logging, data validation, and finally handling the actual request. Using the Chain of Responsibility pattern, you can decouple these operations and create a chain where each operation is handled by a separate object. Show your proposed implementation.

Strategy Design Pattern

- A program to send messages using multiple messaging services

```
public class MessageService {  
    public void send(String type, String msg){  
        if(type.equals(anObject: "WeChat")){  
            System.out.println(x: "Sending message using WeChat");  
            // WechatClient client = getClient();  
            // .....  
            // System.out.println("Failed ending message");  
            // }  
        } else if(type.equals(anObject: "WhatsApp")){  
            System.out.println(x: "Sending message using WhatsApp");  
            // Twilio.init(ACCOUNT_SID, AUTH_TOKEN);  
            // .....  
            // .create();  
        }  
    }  
}
```

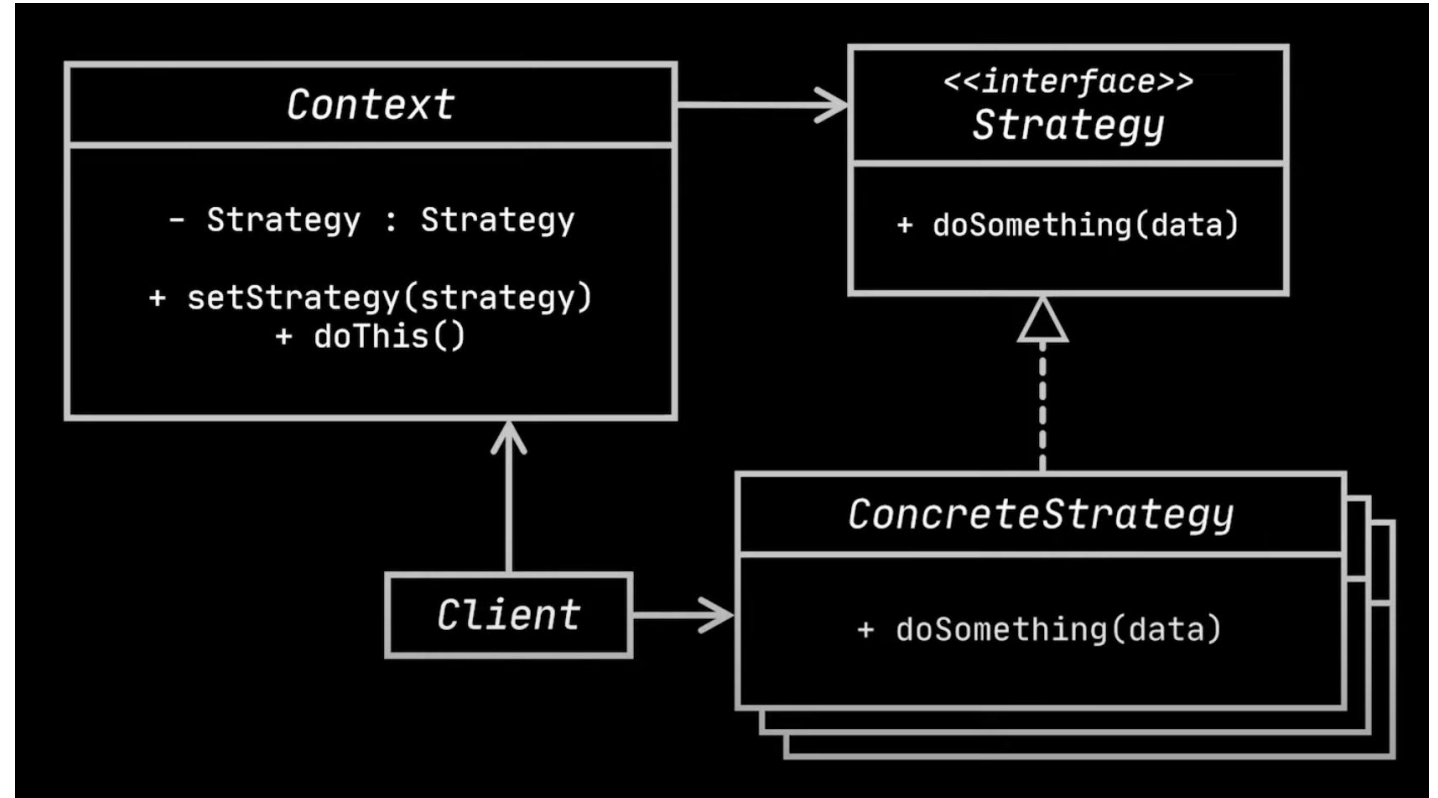
- What are the problems with this code?

Strategy Design Pattern

- Strategy pattern deals with the change in class behavior at runtime. The objects consist of strategies and the context object judges the behavior at runtime of each strategy
- Place each messaging type into its own class to achieve single responsibility principle.
- These classes can easily interchangeable without modification.

Key Components

- Context: Maintains a reference to a concrete strategy and communicate with it via interface.
- Strategy interface: This is an interface common to different algorithms.
- Concrete strategy: Provide its own implementations for the interface.
- Client: Creates a specific strategy object and passes it to the context



Strategy Design Pattern – Ecommerce Example

- Let's take an example of a payment system in an e-commerce application:
- In such an application, customers may have different preferred methods of payment; some may like to pay with credit cards, others with debit cards, PayPal, bank transfer, or even cryptocurrencies. Trying to manage all these payment methods within a single class would make your code complex and harder to maintain.
- With the Strategy pattern, you can define a common interface for all payment methods, with each payment method implemented as a separate class (Concrete Strategy). The Context class (in this case, the shopping cart or payment processing class) can then use this interface to delegate the payment processing task to the payment method selected by the user.

ECommerce Example

```
1 interface PaymentStrategy {
2     void pay(double amount);
3 }
4
5 class CreditCardPaymentStrategy implements PaymentStrategy {
6     @Override
7     public void pay(double amount) {
8         // Process credit card payment
9     }
10 }
11
12 class PayPalPaymentStrategy implements PaymentStrategy {
13     @Override
14     public void pay(double amount) {
15         // Process PayPal payment
16     }
17 }
18
19 // Other payment strategies ...
```

```
21 class ShoppingCart {
22     private PaymentStrategy paymentStrategy;
23
24     public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
25         this.paymentStrategy = paymentStrategy;
26     }
27
28     public void checkout(double amount) {
29         paymentStrategy.pay(amount);
30     }
31 }
32
33 // Usage:
34 public class ECommerce{
35     Run | Debug
36     public static void main(String[] args) {
37         ShoppingCart cart = new ShoppingCart();
38         cart.setPaymentStrategy(new PayPalPaymentStrategy());
39         cart.checkout(amount:100.0);
40     }
41 }
```

Observation

- Open/Closed Principle
- Open for extension: We can add more payment strategy by implementing *PaymentStrategy* interface.
- Close for modification: We do not modify add existing code except passing in new strategy object.
- In the software industry, Open/Closed principle is mostly implemented through Strategy Pattern.

Industry practice – Object Injection

```
public interface DataExporter {  
    void export(Data data);  
}
```

```
public class PdfExporter implements DataExporter {  
    public void export(Data data) { /* PDF Logic */ }  
}  
  
public class CsvExporter implements DataExporter {  
    public void export(Data data) { /* CSV Logic */ }  
}
```

```
public class ExportService {  
    private final DataExporter exporter;  
  
    // The specific exporter is "injected" at runtime  
    public ExportService(DataExporter exporter) {  
        this.exporter = exporter;  
    }  
  
    public void executeExport(Data data) {  
        exporter.export(data); // Open for any implementation!  
    }  
}
```

Benefits

- **Flexibility:** You can alter the application's behavior at runtime by replacing the strategy object within the context object.
- **Encapsulation:** Related algorithms are grouped into strategy classes, which encapsulate complex behavior that could otherwise clutter other classes.
- **Reusability:** Since the strategy classes are decoupled from the context, they can be reused across different contexts.
- **Maintainability:** The Strategy Design Pattern makes the code more maintainable. Changes to one strategy do not affect others. Adding a new strategy or updating an existing one involves changes only to the strategy class itself, not the context class.
- **Testing:** Each strategy can be tested independently of the context and other strategies, which makes unit testing easier and more robust

Application of Strategy Design Pattern

- The Strategy design pattern shines in scenarios where an object's behavior or algorithm can change dynamically at runtime. It's especially useful when you have several variants of an algorithm or operation and the exact variant to be used isn't known until runtime.

Exercise 3

Courier Service Problem

- The cost calculation for courier service is base on the type of item to be sent. Currently there are 2 type of items can be sent using the courier service, namely Parcel and Document.
- The algorithm to calculate the cost of sending Parcel is first calculate using algorithm A which is base on the weight, and then calculate using algorithm B which is base on the dimension. The final cost is the cost from either algorithm A or B depending on whichever is higher.
- The algorithm to calculate the cost of sending Document is algorithm C.
- In your problem, algorithm A, B, and C is negligible.
- Write a program following Strategy Design Pattern for the problem.

Exercise 4

- You are tasked with designing a flexible sorting system for a data analysis application. The application needs to sort lists of data in various ways, such as by numerical value, alphabetical order, or custom-defined criteria. The sorting method should be easily interchangeable during the run-time without modifying the existing code structure.
- **Task:**
 - Describe the Strategy design pattern and explain how it can be applied to solve the problem of interchangeable sorting methods.
 - Draw a class diagram to represent your design for the system. Include at least three different sorting strategies: numerical, alphabetical, and a custom strategy of your choice.
 - Briefly explain the advantages of using the Strategy pattern in this context.

Answer

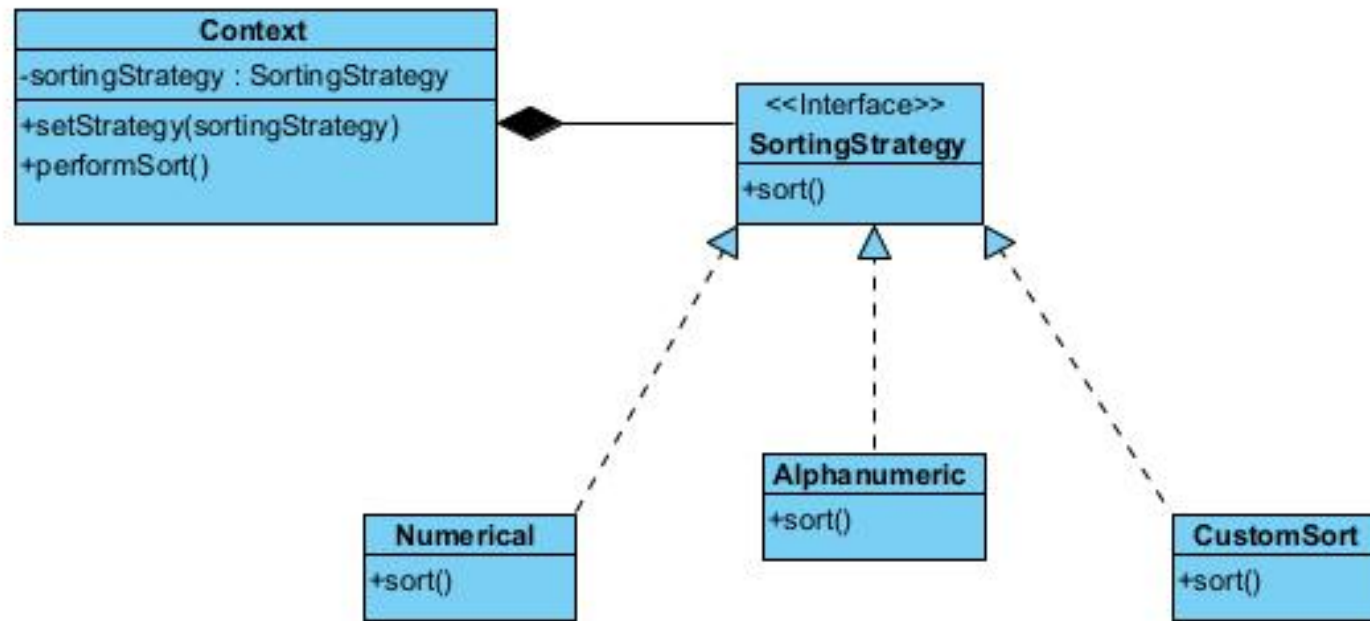
- **Description of Strategy Pattern:**

- The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. This pattern allows the algorithm to vary independently from the clients that use it.

- **Advantages of Using the Strategy Pattern:**

- **Flexibility:** It allows the client to choose and change the sorting algorithm at runtime without altering the client code.
- **Maintainability:** New sorting strategies can be added easily without modifying existing classes, adhering to the Open/Closed Principle.
- **Reusability:** Different sorting strategies are encapsulated in their own classes, making them reusable across different context.

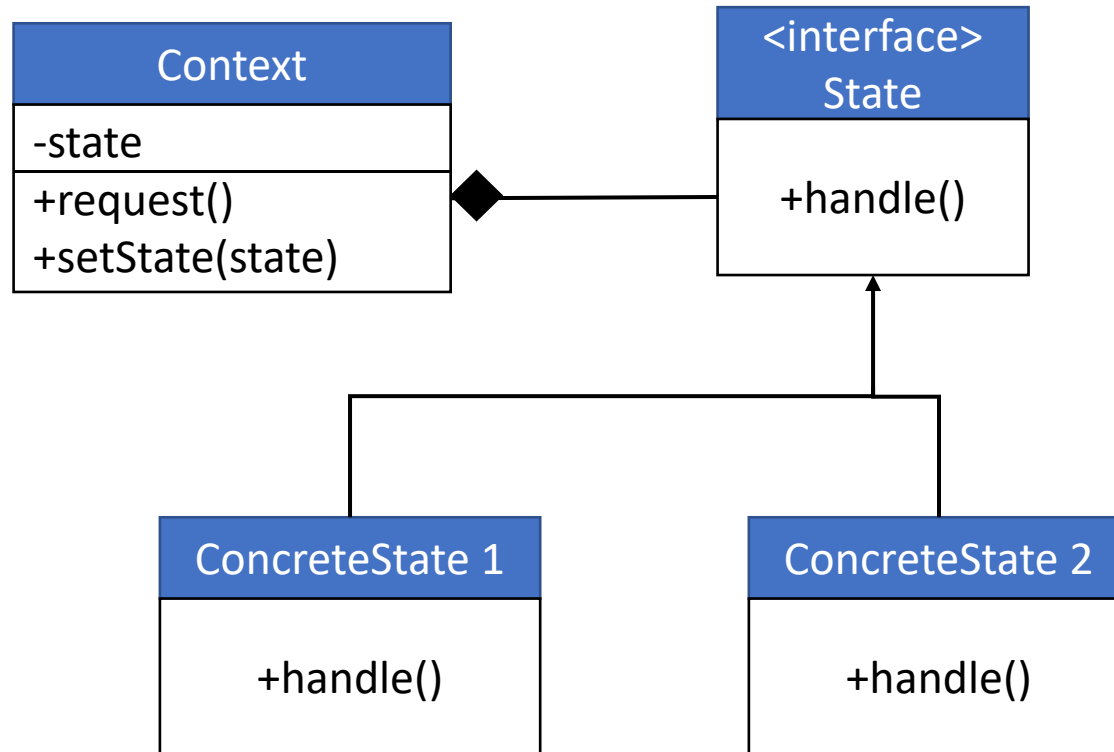
Answer



State Pattern

- The State pattern is a **behavioral** pattern - it's used to manage algorithms, relationships and responsibilities between objects.
- The definition of State provided in the original Gang of Four book on Design Patterns states: *Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.*

State Pattern – The Structure



Key Components

- **Context** stores a reference to one of the concrete state objects and delegates to it all the state-specific work. The context communicates with the state object via the state interface. The context exposes a setter for passing it a new state object.
- The **State** interface defines a common interface for all concrete states, encapsulating all behavior associated with a particular state.
- **Concrete States** provide their own implementations for the state-specific methods. To avoid duplication of similar code across multiple states, you may provide intermediate abstract classes that encapsulate some common behavior.
- Both context and concrete states can set the next state of the context and perform the actual state transition by replacing the state object linked to the context.

MP3 Example

```
1  public class MP3PlayerContext {  
2      private State state;  
3  
4      public MP3PlayerContext(State state) {  
5          this.state = state;  
6      }  
7  
8      public void play() {  
9          state.pressPlay(this);  
10     }  
11  
12     public void setState(State state) {  
13         this.state = state;  
14     }  
15  
16     public State getState() {  
17         return state;  
18     }  
19 }
```

```
1 interface State {  
2     public void pressPlay(MP3PlayerContext context);  
3 }
```

```
1 public class StandbyState implements State {  
2     public void pressPlay(MP3PlayerContext context) {  
3         System.out.println("Playing...");  
4         context.setState(new PlayingState());  
5     }  
6 }
```

```
1 public class PlayingState implements State {  
2     public void pressPlay(MP3PlayerContext context) {  
3         System.out.println("I am already in PLAYING state");  
4     }  
5 }
```

```
1 public class PlayMusic {  
    Run | Debug  
2     public static void main(String[] args) {  
3         MP3PlayerContext player = new MP3PlayerContext(new StandbyState());  
4  
5         player.play();  
6     }  
7 }  
8
```

Workflow System

- One complex scenario where State Design Pattern can be beneficial is a workflow system. A workflow includes a number of processes that usually occurs in a sequence, where each process has its own set of rules, responsibilities, and the transition to the next process.
- For instance, let's consider a Document Approval System where a document goes through several states:
 - Draft, Review, Approve, Disapprove

Exercise 5

- Draw a class diagram for the following workflow program.

```
1 public interface State {  
2     void review(Document doc);  
3     void approve(Document doc);  
4     void disapprove(Document doc);  
5 }
```

```
class Draft implements State {  
  
    @Override  
    public void review(Document doc){  
        System.out.println(x:"Start to review the document");  
        doc.setState(new Review());  
    }  
  
    @Override  
    public void approve(Document doc){  
        System.out.println(x:"The document is not being reviewed yet, please review the document before approve");  
    }  
  
    @Override  
    public void disapprove(Document doc){  
        System.out.println(x:"The document is not being reviewed yet, please review the document before disapprove");  
    }  
}
```

```
class Review implements State {
```

```
    @Override
```

```
    public void review(Document doc){
```

```
        System.out.println(x:"The document is in the middle of reviewing.");
```

```
    }
```

```
class Approved implements State {
```

```
    @Override
```

```
    public void review(Document doc){
```

```
        System.out.println(x:"The document is already approved.");
```

```
    }
```

```
    @Override
```

```
    public void
```

```
        System
```

```
    }
```

```
    @Override
```

```
    public void
```

```
        System
```

```
        doc.s
```

```
    }
```

```
}
```

```
class Disapproved implements State {
```

```
    @Override
```

```
    public void review(Document doc){
```

```
        System.out.println(x:"The document is already approved.");
```

```
    }
```

```
    @Override
```

```
    public void approve(Document doc){
```

```
        System.out.println(x:"You have approved the previously disapproved document.");
```

```
        doc.setState(new Approved());
```

```
    }
```

```
    @Override
```

```
    public void disapprove(Document doc){
```

```
        System.out.println(x:"The document is already disapproved.");
```

```
    }
```

```
}
```

```
class Document {  
  
    private State state;  
  
    public Document() {  
        state = new Draft(); // initial state  
    }  
  
    public void setState(State state) {  
        this.state = state;  
    }  
  
    public void review(){  
        state.review(this);  
    }  
  
    public void approve(){  
        state.approve(this);  
    }  
  
    public void disapprove(){  
        state.disapprove(this);  
    }  
}
```

```
public class Workflow {  
    Run | Debug  
    public static void main(String[] args)  
    {  
        Document document = new Document();  
  
        // Edit the document here  
  
        document.review();  
  
        // Review the document here  
  
        document.approve();  
  
        document.approve();  
  
        document.review();  
  
        document.disapprove();  
    }  
}
```

Benefits

- Organized Control Flow: The State pattern can improve control flow if a system often switches between a number of states in a complex manner.
- Localize State-Specific Behavior: All state-related operation is coded into separate classes. This approach allows you to group the states' behaviors within corresponding state class itself.
- State Transitions Explicit: State transitions can be made more explicit and predictable. The code is easier to read and understand.
- Extensibility: New states can be added easily by just creating new state classes.
- Simplify Code: The State Pattern helps to clean up the code by eliminating large monolithic conditional blocks.

Application of State Pattern

- Use the State pattern when you have an object that behaves differently depending on its current state, the number of states is enormous, and the state-specific code changes frequently.
- Use the pattern when you have a class polluted with massive conditionals that alter how the class behaves according to the current values of the class's fields.
- Use State when you have a lot of duplicate code across similar states and transitions of a condition-based state machine.

Exercise 6

A traffic light commonly has three states:

1. Red: Vehicles must stop.
2. Green: Vehicles can go.
3. Yellow: Vehicles should prepare to stop, as it will turn red soon.

Here are the behaviors and transitions for these states:

- In the Red state, after a certain period, the state transitions to Green, allowing vehicles to go.
- In the Green state, after a certain period or if emergency conditions are noticed, the state transitions to Yellow as a signal to prepare for a stop.
- In the Yellow state, after a brief period, the state transitions to Red, indicating the vehicles must stop.

The Traffic Light object (Context) can contain a state reference to manage its current Traffic Light State (State). The State interface or abstract class declares the method `changeLight()`, which changes the state of the traffic light object. The concrete state classes (Red, Green, Yellow) implement the `changeLight()` function to dictate how state changes.

Solve the problem using State design pattern.